Warranty Vault



SE4041 Assignment 02 – part A

Technical Documentation

Student name : Hitiarachchi A M L

Student ID – IT22079640

1. Overview and scope

Warranty Vault is an iOS app for recording product purchases and tracking warranty expiry. It imports receipt images or PDFs, recognises text locally, parses likely purchase fields, persists warranty items, and schedules local reminders. This document describes the known architecture and implementation based on the project facts supplied.

2. Technology stack and requirements

Area	Technology / details
Language and UI	Swift; SwiftUI
Persistence	SwiftData, built on Core Data
OCR	Vision: VNRecognizeTextRequest, accurate mode, language correction
Text and categories	NaturalLanguage; CategoryKeywords.json
Charts and notifications	Swift Charts; UserNotifications
Import and sharing	PhotosPicker; fileImporter; ShareLink
Localization	String Catalogs for English and Sinhala
Development	Xcode 16.4; minimum iOS 17.6
Test environment	iPhone 16 simulators running iOS 18.1 and 18.6
Architecture	MVVM; repository pattern; protocol-based service abstractions

3. Project structure

The following is a conceptual layer map based on the supplied project description. Confirm folder names against the repository before treating this as an exact filesystem tree.

```
WarrantyVault/
├── Models/        WarrantyItem, ItemCategory, WarrantyStatus, ParsedReceipt
├── ViewModels/    AddItemViewModel, InsightsSummary, SettingsViewModel
├── Views/         Onboarding, Vault, Item Detail, Add Flow, Insights, Settings
├── Components/    ExpiryRing, CardStyle, VaultWarrantyCard
├── Services/      OCRService, ReceiptParser, CategorySuggester,
│                  WarrantyReminderScheduling, WarrantyRepository
├── Utilities/     CurrencyFormatter, WarrantyCSV, WarrantySummary
├── Resources/     CategoryKeywords.json, String Catalogs, sample receipt
├── Tests/         Unit tests and XCTest UI tests
└── docs/          Screenshots and project documentation
```

4. Architecture

The app uses MVVM to separate view presentation from application logic. Views bind to view models. The repository pattern centralises access to ModelContext, and protocols abstract text recognition and reminder scheduling. The reminder service has a UserNotification implementation and a mock used by tests.

5. Data model

WarrantyItem is the main persisted model. It includes the purchase and warranty information needed to calculate a computed expiryDate and a WarrantyStatus. ItemCategory includes phone, laptop, appliance, furniture, vehicle, and other. WarrantyStatus represents active, expiring soon (within 30 days), or expired.

Conceptual relationship diagram (Mermaid source; confirm individual stored fields against the source code):

6. Receipt processing pipeline

The user selects an image through PhotosPicker or a document through fileImporter. A receipt image/PDF handler prepares the content; PDF rendering is capped at a scale of min(2, 1600 / longest side), and expensive decoding/rendering is performed off the main thread. OCRService uses VNRecognizeTextRequest in accurate mode with language correction. ReceiptParser extracts the date and total and returns parsed values with per-field confidence. CategorySuggester uses keyword data to suggest a category. The user reviews the form, corrects values if needed, chooses a warranty period, and saves the item. The app then schedules reminders.

7. Parsing rules

7.1 Dates

The parser uses label-first search for invoice date, bill date, transaction date, purchased, date, and dt, including a possible label on the preceding line. If no labelled date is found, it searches for date patterns. Supported formats include dd/MM/yyyy, dd-MM-yyyy, dd.MM.yyyy, two-digit years, yyyy-MM-dd, yyyy/MM/dd, and month names. It accounts for times attached to dates, spaces inside dates, and OCR confusions such as O/0, l/I/1, and S/5. Future dates and dates over 15 years old are rejected; tied candidates receive low confidence.

7.2 Total amount

The parser prioritises labels such as grand total, net total, amount due, and total. If a label has no number on the same line, the next line is checked. Cash, change, tendered, VAT, tax, discount, subtotal, and quantity lines are excluded as total candidates. If cash minus change matches a candidate total, the match is treated as supporting evidence. A bug where a change value was selected instead of the total was fixed by applying these rules.

8. Navigation and interface

AppRouter stores a NavigationPath per tab and uses a typed Route enum. The main screens include onboarding, vault, item detail, add flow, insights, and settings. The vault supports list/grid views, search, filters, swipe

actions, and a context menu. Item detail supports pinch zoom, double-tap, and sharing. ExpiryRing is a custom animated component, and CardStyle and VaultWarrantyCard provide reusable styling.

9. Notifications

WarrantyReminderScheduling abstracts reminder delivery. The production implementation uses UserNotifications, while a mock scheduler supports unit tests. Reminders are scheduled 30, 7, and 1 day before expiry. Settings provide a reminders toggle, a five-second test notification, and a Scheduled reminders list. The future calendar notifications are not observed live during short testing; verification uses the pending list and mock-scheduler unit tests.

10. Performance design

- Image decoding and PDF rendering are performed off the main thread using Task.detached.
- PDF render scale is capped at min(2, 1600 / longest side).
- Receipt thumbnails are cached with NSCache at 320 pixels.
- The detail image is decoded once into @State.
- Insights calculations are performed in one pass.
- Single-item lookups use fetchLimit = 1.

11. Accessibility and localization

The app uses String Catalogs for English and Sinhala interface strings. CurrencyFormatter formats values such as Rs. 129,900.00 and provides a VoiceOver-friendly spoken currency description. The interface includes visual status cues and an expandable raw-text area. Sinhala receipts themselves have not been tested, so interface localization should not be interpreted as verified Sinhala OCR support.

12. Testing strategy

Unit tests cover ReceiptParser, CategorySuggester, repository behaviour, reminder scheduling through a mock, AddItemViewModel, CSV export, summary calculations, and currency formatting. XCTest UI tests cover onboarding, manual item creation, filtering, and deletion. Supported launch arguments include -skipOnboarding, -resetOnboarding, -resetVault, and -seedVault.

For receipt accuracy testing, used ten receipts and compare extracted values with the text actually printed on each image. Record store, date, amount, and category accuracy separately. For user testing, asked three participants to add an item from a receipt and locate the item expiring soonest, then record a 1–5 ease rating and one comment.

13. Known issues and limitations

- No live camera capture; receipts are imported from Photos or Files.
- OCR accuracy depends on receipt quality, layout, and language.
- Sinhala receipts are untested.

- Real 30/7/1-day reminders cannot be verified by waiting through the full warranty timeline; pending notifications and mock-scheduler tests are used.
- The app was tested on simulators; physical-device testing and TestFlight distribution were limited by the development setup.

14. Possible extensions

Potential extensions include live camera capture, broader real-world receipt testing, physical-device notification testing, improved support for local receipt layouts and languages, and additional accessibility validation. Any new feature should be added with corresponding unit and UI tests.